

平成 28 年度

筑波大学大学院博士課程
システム情報工学研究科
コンピュータサイエンス専攻

博士前期課程（一般入学試験 2 月期）

試験問題 **基礎科目（情報基礎）**

Fundamentals of Computer Science

[注意事項][Instructions]

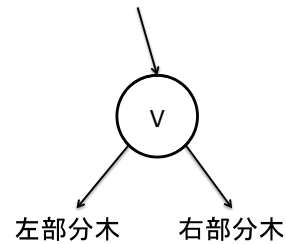
1. 試験開始の合図があるまで、問題の中を見てはいけません。また、筆記用具を手に持ってはいけません。
Do NOT open this booklet before the examination starts. In addition, do not have a pen in hand.
2. 試験開始の合図のあとで、全ての解答用紙の定められた欄に、研究科、専攻、受験番号を記入すること。
Fill in the designated spaces on each answer sheet with the name of the graduate school, name of your main (major) field, and the examination number after the examination starts.
3. この問題は全部で 10 ページ（表紙を除く）です。1～5 ページは日本語版、6～10 ページは英語版です。
This booklet consists of 10 pages, excluding the cover sheet. The Japanese version is shown on pages 1-5 and the English version on pages 6-10.
4. 解答用紙（罫線有り）を 2 枚、下書き用紙（白紙）を 1 枚配布します。
You are given two ruled answer sheets and one white draft sheet.
5. 問題は全部で 2 問あります。問題 I の解答を 1 枚目、問題 II の解答を 2 枚目の解答用紙に、必ず分けて記入すること。
There are two problems. Write the answers to Problem I on the first answer sheet and the answers to Problem II on the second answer sheet.
6. 解答用紙に解答を記述する際に、問題 I の解答か問題 II の解答かを必ず明記すること。
When writing the answers, clearly label the problem number on each answer sheet.

平成 28 年 2 月 1 日

問題 I の解答は 1 枚目の解答用紙に記入すること。問題 I の解答であることを明記すること。

問題 I 整数の集合を保持する 2 分探索木を考える。そのノードを、C 言語で以下の構造体として定義する。

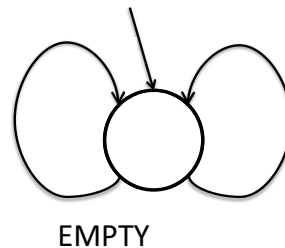
```
typedef struct tree_node {
    int value;
    struct tree_node *left, *right;
} *tree;
```



left が指す左の部分木には、value の値が V より小さい値を持つノードのみが含まれる。また、right が指す右の部分木には、V より大きい値を持つノードのみが含まれる。

空の木 (ノードを 1 つも持たない木) は、特別なノード EMPTY を指すポインタで表現する。

```
struct tree_node EMPTY = {
    -1, /* dummy value */
    &EMPTY, /* left */
    &EMPTY /* right */
};
```



この約束に従い、2, 5, 11, 13, 28 の整数の集合を保持する 2 分探索木の例 (T1 と呼ぶ) を図 1 に示す。

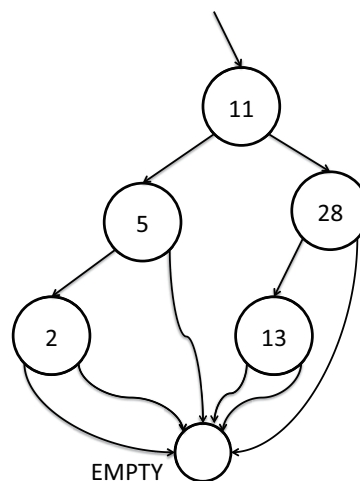


図 1: 2 分探索木の例 (T1)

- (1) 次の関数 `find_node` を考える.

```
tree find_node(tree t, int v) {
    while (v != t->value) {
        if (v < t->value) {
            t = t->left;
        } else {
            t = t->right;
        }
    }
    return t;
}
```

この関数 `find_node` に、図 1 の木 `T1` と、`T1` に含まれない値 9 を引数として、`find_node(T1, 9)` のような関数呼び出しを行ったとする. `EMPTY` の `value` フィールドの値は -1 であるとする. この時、以下のうち正しい文の記号を 1 つ選んで答えなさい.

- (a) `find_node` は、`value` フィールドの値として 9 を持つノードへのポインタを返す.
 - (b) `find_node` は、何も指し示していないことを表す `NULL` ポインタを返す.
 - (c) `find_node` は、`EMPTY` へのポインタを返す.
 - (d) `find_node` は、停止しない.
- (2) 以下の関数 `contains` は、`find_node` を用いて、2 分探索木の中に値 `v` があれば 0 でない値 (真) を、なければ 0 (偽) を返す関数である.

```
int contains(tree t, int v) {
    EMPTY.value = v;
    tree node = find_node(t, v);
    return (A) ;
}
```

(A) に入る適切な式を答えなさい.

- (3) n 個の異なる整数をランダムな順序で挿入して得られる 2 分探索木を第 1 引数として `contains` を呼び出した時、漸近的な平均実行時間はどうか. 以下のうち正しい文の記号を 1 つ選んで答えなさい.
- (a) $\log n$ に比例する. ただし、 $\log$ は自然対数を表す.
 - (b) n に比例する.
 - (c) $n \log n$ に比例する. ただし、 $\log$ は自然対数を表す.
 - (d) n^2 に比例する.
 - (e) 上のいずれでもない.

(4) `EMPTY` 以外のノード数が n 個である 2 分探索木を第 1 引数として `contains` を呼び出した時、最悪の場合の漸近的な実行時間はどうか。以下のうち正しい文の記号を 1 つ選んで答えなさい。

- (a) $\log n$ に比例する。ただし、 $\log$ は自然対数を表す。
- (b) n に比例する。
- (c) $n \log n$ に比例する。ただし、 $\log$ は自然対数を表す。
- (d) n^2 に比例する。
- (e) 上のいずれでもない。

(5) 2 分探索木 `t` に、値 `v` を挿入する関数 `insert` を考える。

```
tree insert(tree t, int v) {
    if (t == &EMPTY) {
        tree node = malloc(sizeof(struct tree_node));
        node->value = v;
        node->left = node->right = &EMPTY;
        return node;
    }
    if (v < t->value) {
        (B) = (C) ;
    } (D) {
        (E) = (F) ;
    }
    return t;
}
```

`malloc` は、引数で与えた大きさのメモリ領域を割り付けて返す関数で、割り付けは必ず成功するものとする。関数 `insert` は、以下のような仕様とする。

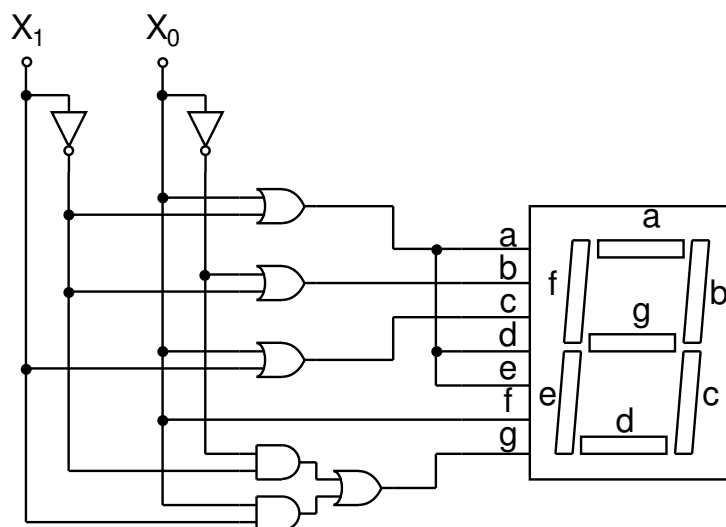
- `t` が空の木 (`EMPTY` へのポインタ) なら、単一のノードを持つ木を作って返す。
- `t` に、`v` が既に含まれていれば、何もしない。
- `t` が空でなく、`v` が含まれていなければ、`v` を含む新しいノードを `t` の適切な場所に挿入し、変更した木を返す。

(B)～(F) を埋めて、関数 `insert` を完成させなさい。

問題 II の解答は 2 枚目の解答用紙に記入すること．問題 II の解答であることを明記すること．

問題 II

- (1) X_1, X_0 を入力とし, a, b, c, d, e, f, g を出力とする以下に示すデジタル回路を考える．この回路の出力端子に, 7 セグメント LED が接続されている．それぞれの LED に, 記号 a, b, c, d, e, f, g が割り当てられている．次の設問に答えなさい．



- (1-1) 次の真理値表を完成しなさい．

X_1	X_0	a	b	c	d	e	f	g
0	0							
0	1							
1	0							
1	1							

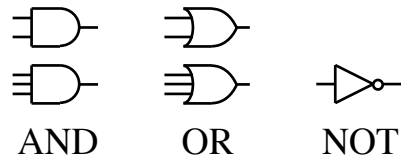
- (1-2) 7 セグメント LED に表示されるパターンを, 以下の形式で答えなさい．LED は 1 のときに点灯し, 0 のときに消灯する．例えば, $(a, b, c, d, e, f, g) = (1, 1, 1, 1, 0, 0, 1)$ のとき, 表示されるパターンは,  となる．

X_1	X_0	パターン
0	0	
0	1	
1	0	
1	1	

- (2) 次のように, X_3, X_2, X_1, X_0 を入力として, Y_3, Y_2, Y_1, Y_0 を出力するデジタル回路について考える.
次の設問に答えなさい.

X_3	X_2	X_1	X_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	0	0	1
0	0	0	1	0	0	1	1
0	0	1	1	0	1	1	1
0	1	1	1	1	1	1	1
1	1	1	1	1	1	1	0
1	1	1	0	1	1	0	0
1	1	0	0	1	0	0	0
1	0	0	0	0	0	0	0
上記以外				P	P	P	P

- (2-1) $P = 0$ とする. 出力 Y_3, Y_2, Y_1, Y_0 のそれぞれについてのカルノー図を示しなさい.
- (2-2) (2-1) のカルノー図を使って, 出力 Y_3, Y_2, Y_1, Y_0 の最簡形の論理式を示しなさい.
- (2-3) (2-2) の論理式を実現する回路図を示しなさい. ただし, 以下の回路記号を必要に応じて用いなさい. AND ゲートと OR ゲートは 2 入力または 3 入力のみとする.

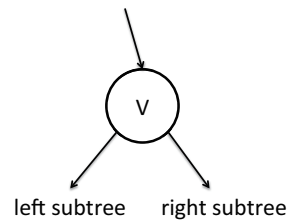


- (2-4) P を don't care とする. このときの最簡形の論理式に基づいた回路図を示しなさい.

Write the answers to Problem I on the first answer sheet, and clearly label it at the top of the page as “Problem I.”

Problem I Let us consider binary search trees (BSTs) which implement sets of integers. Suppose we define a structure of C language for nodes of such BSTs.

```
typedef struct tree_node {
    int value;
    struct tree_node *left, *right;
} *tree;
```



The left subtree (pointed to by `left`) only contains nodes with `value` smaller than `V`, and the right subtree (pointed to by `right`) contains nodes with `value` larger than `V`.

An empty tree (that is, a tree without node) is represented by a pointer to a special node `EMPTY`.

```
struct tree_node EMPTY = {
    -1, /* dummy value */
    &EMPTY, /* left */
    &EMPTY /* right */
};
```

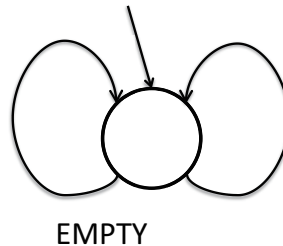


Figure 1 illustrates an instance of BST, named `T1`, which contains set of integers 2, 5, 11, 13, and 28, following the convention above.

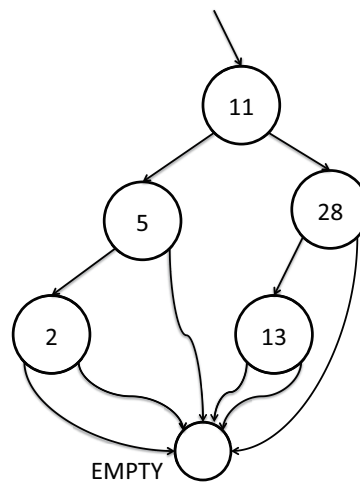


Fig. 1: An example BST (`T1`)

- (1) Let us consider the function `find_node` below:

```
tree find_node(tree t, int v) {
    while (v != t->value) {
        if (v < t->value) {
            t = t->left;
        } else {
            t = t->right;
        }
    }
    return t;
}
```

Suppose we call the function `find_node` with T1 in Figure 1 and 9 (which is not in T1) as arguments. Assume the `value` field of `EMPTY` is -1. Choose the correct statement from below and answer with its letter.

- (a) `find_node` returns a node which has 9 in the `value` field.
 - (b) `find_node` returns NULL pointer, which is a special value indicating the pointer points to nothing.
 - (c) `find_node` returns a pointer to `EMPTY`.
 - (d) `find_node` never terminates.
- (2) The following function `contains` uses `find_node` to look up a BST and returns a non-zero value (true) if the BST contains `v` and returns 0 (false) otherwise.

```
int contains(tree t, int v) {
    EMPTY.value = v;
    tree node = find_node(t, v);
    return (A) ;
}
```

Find an appropriate expression for (A).

- (3) When we call `contains` giving a BST which can be obtained by inserting n distinct integers in random order as its first argument, then what will be the asymptotic average-case running time? Choose the correct statement from below and answer with its letter.
- (a) The running time is proportional to $\log n$, where $\log n$ denotes the natural logarithm of n .
 - (b) The running time is proportional to n .
 - (c) The running time is proportional to $n \log n$, where $\log n$ denotes the natural logarithm of n .
 - (d) The running time is proportional to n^2 .
 - (e) None of the above.

- (4) When we call `contains` giving a BST with n nodes (excluding `EMPTY` node) as its first argument, then what will be the asymptotic worst-case running time? Choose the correct statement from below and answer with its letter.
- (a) The running time is proportional to $\log n$, where $\log n$ denotes the natural logarithm of n .
 - (b) The running time is proportional to n .
 - (c) The running time is proportional to $n \log n$, where $\log n$ denotes the natural logarithm of n .
 - (d) The running time is proportional to n^2 .
 - (e) None of the above.
- (5) Consider the function `insert` below, which inserts v in a BST t .

```
tree insert(tree t, int v) {
    if (t == &EMPTY) {
        tree node = malloc(sizeof(struct tree_node));
        node->value = v;
        node->left = node->right = &EMPTY;
        return node;
    }
    if (v < t->value) {
        (B) = (C) ;
    } (D) {
        (E) = (F) ;
    }
    return t;
}
```

Here, `malloc` is a function which allocates and returns a memory area specified by its argument. We assume the allocation always succeeds.

The specification of `insert` is as follows.

- If t is an empty tree (i.e. a pointer to `EMPTY`), then the function will create a singleton tree and return it.
- If t already contains v , the function will do nothing.
- If t is not empty and t does not contain v , then the function will insert a new node, which contains v , in the appropriate position of t and return the modified tree.

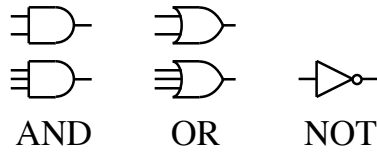
Fill in blanks (B) to (F) to complete the function.

Problem II

- (2) Let us consider a digital circuit with X_3, X_2, X_1 and X_0 as inputs and with Y_3, Y_2, Y_1 and Y_0 as outputs. Answer the following questions.

X_3	X_2	X_1	X_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	0	0	1
0	0	0	1	0	0	1	1
0	0	1	1	0	1	1	1
0	1	1	1	1	1	1	1
1	1	1	1	1	1	1	0
1	1	1	0	1	1	0	0
1	1	0	0	1	0	0	0
1	0	0	0	0	0	0	0
others				P	P	P	P

- (2-1) Let $P = 0$. Show the Karnaugh maps for the outputs Y_3, Y_2, Y_1 and Y_0 .
- (2-2) Describe the simplest logical expressions for each of the outputs Y_3, Y_2, Y_1 and Y_0 from the Karnaugh maps in question (2-1).
- (2-3) Derive a schematic for realizing the logical expressions in question (2-2). Here, use the following symbols if necessary. AND and OR logic gates have two input terminals or three input terminals only.



- (2-4) Let P be “don’t care.” Derive a schematic based on the simplest logical expression.

